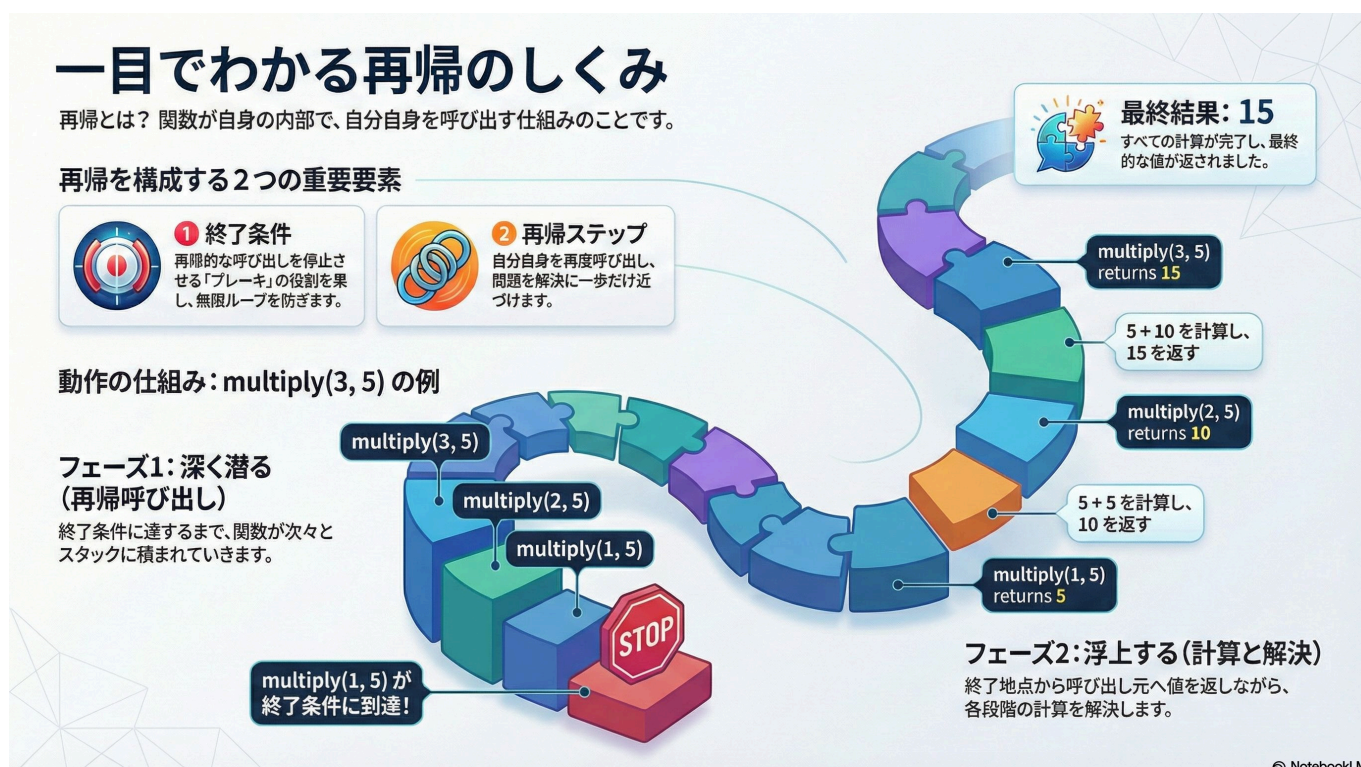


再帰



1. はじめに：再帰的思考への第一歩

本章は、プログラム初心者の皆さんに、重要なプログラミング概念である「再帰 (Recursion)」の基礎を解説することを目的としています。再帰は単なるテクニックではなく、複雑な問題をより小さな、自己相似的な問題に分割して考える一種の「思考法」です。

再帰の理解が不可欠なのは、それが多くの高度な課題に対するエレガントな解決策を提供するからです。例えば、連結リストや二分木といったデータ構造の探索、あるいはチェスのようなゲームで無数のシナリオを探索する際など、再帰は反復処理では複雑になりがちなロジックを、驚くほどシンプルかつ直感的に記述することを可能にします。このガイドを通じて、再帰の基本原則から具体的なコード実装までを段階的に学び、その強力な問題解決能力を身につけていきましょう。

2. 再帰 (Recursion) の基本概念

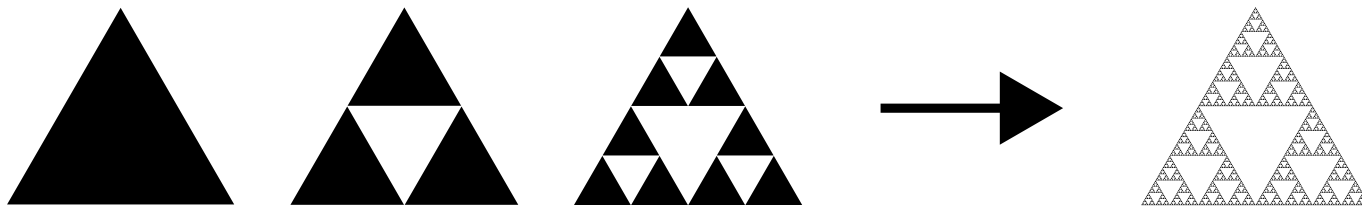
このセクションでは、再帰の理論的な基盤を固めます。再帰とは何か、どのような利点と注意点があり、どのような場面で活用されるのかを明確に理解することが、効果的な実装への第一歩となります。

2.1. 再帰とは何か？

プログラム言語における再帰とは、関数が自身の内部で自分自身を呼び出すことを指します。これにより、同じロジックを異なるデータに対して繰り返し適用することが可能になります。

「再帰」のイメージをつかむための、参考として、同じロジックを図形に繰り返し適用する例を、[Wikipediaのページ](#)を参考にして、見てみましょう。

下の図は、「シェルピンスキーのギャスケット」と呼ばれている図形を作る過程です（実際には無限回の繰り返しが必要なため、近似的な作画です）。



(Twisp - 投稿者自身による著作物, パブリック・ドメイン, <https://commons.wikimedia.org/w/index.php?curid=3402690>による)

1. 正三角形から始めます
2. それを大きさも形も同じ4つの正三角形に分割し、中央の三角形を取り除きます
3. その結果出来た小さな三角形すべてに対して、手順2を無限に繰り返します

引用元:

- https://en.wikipedia.org/wiki/Fractal_dimension
- https://en.wikipedia.org/wiki/Sierpi%C5%84ski_triangle

このように **ロジックをある対象に適用し、その結果に対して、さらに同じロジックを適用する** の繰り返しですが、再帰です。

ただし、上のような自己相似的な図形（フラクタル図形と呼ばれます）と、C言語の再帰関数には、決定的な違いがあります。それは、フラクタル図形を形成するための繰り返しは理論上は無限に行われるのに対し、**再帰関数には必ず終了条件があること** です。フラクタル図形が片道切符だとしたら、再帰関数は目的地までの往復切符なのです。

2.2. 再帰の利点と注意点

次に、利点と注意点をまとめましょう。再帰は強力なツールですが、その特性を理解して適切に使う必要があります。その二面性を以下の表にまとめます。

利点 (Advantage)	注意点 (Caution)
非常にすっきりと洗練された、直感的に理解しやすいコードを記述できる場合があります。	再帰が深くなりすぎた場合に、非常に大量のメモリが使用される可能性があります。
解説：問題の構造とコードの構造が一致するため、ロジックが明確になり、可読性が向上します。	解説：関数呼び出しごとにメモリ（スタック領域）が消費されるため、終了条件が不適切だとメモリを使い果たし、プログラムがクラッシュする原因となります。

2.3. 主な使用例

再帰は特に以下のようなシナリオでその真価を発揮します。

1. 連結リストや二分木などの再帰的なデータ構造の探索

連結リストや二分木といったデータ構造（これらについては、次章と次次章で学びます）は、部分が全体と同じ構造を持つ「自己相似的」な性質を持っています。そのため、構造を辿る処理を再帰で記述すると非常に自然で簡潔になります。

2. チェスのようなゲームにおける可能性のあるシナリオの探索

ある局面から次の一手を考え、その手によって生じる新たな局面でまた次の一手を考える、というプロセスは本質的に再帰的です。これにより、膨大な可能性のツリーを効率的に探索できます。

再帰が「何であるか」を理解したところで、次にそれが「どのように機能するのか」という仕組みを詳しく見ていきましょう。

3. 再帰を構成する2つの重要要素

全ての再帰関数は、安全かつ正しく機能するために不可欠な2つの構成要素から成り立っています。それが

- 再帰ステップ: 再帰的に関数（自分自身）を呼び出す。
- 終了条件: これ以上、自分自身を呼び出すことなく、値を返す（折り返し地点）

です。これらの要素を理解することが、再帰をマスターする鍵となります。

3.1. 再帰ステップ（Recursive Step）

再帰ステップは、関数が自分自身を再度呼び出す部分です。ここでの最も重要な原則は、再帰呼び出しを行うたびに、必ず問題が終了条件に一步近づくように状態を変化させなければならないという点です。例えば、処理対象の数値を減らしたり、データ構造の次の要素に進んだりすることで、最終的に終了条件に到達するように設計します。

3.2. 終了条件（Terminating Case）

終了条件は、再帰がいつ終了するかを定義する条件です。この条件が満たされたとき、関数は自分自身をそれ以上呼び出すことなく、値を返して呼び出し元に戻ります。これは、無限に関数を呼び出し続けてメモリを枯渇させてしまう「無限再帰」を防ぐための、極めて重要な安全装置として機能します。

これら2つの要素が連携することで、複雑な問題を段階的に単純化し、最終的に解にたどり着くという、安全で効果的な再帰処理が実現されるのです。

4. コードで学ぶ再帰：乗算関数の徹底解説

これまでに学んだ理論的知識を、具体的なC言語のコードに適用してみましょう。ここでは、その動作を徹底的に分析するために、再帰的な加算を用いて乗算を実現するmultiply関数を取り上げます。この関数は、例えば 5×3 をあたかも $5 + 5 + 5$ のように計算します。

疑似言語

```

○整数型: multiply(整数型 x, 整数型 y)
    /* 終了条件 */
    if (xが1と等しい)
        return y;
    /* 再帰ステップ */
    elseif (xが1より大きい)
        return y + multiply(x-1, y);
    endif

○main()
    multiply(3, 5)の結果を表示

```

C言語

```

#include <stdio.h>

unsigned int multiply(unsigned int x, unsigned int y) {
    /* 終了条件 */
    if (x == 1) {
        return y;
    /* 再帰ステップ */
    } else if (x > 1) {
        return y + multiply(x-1, y);
    }
}

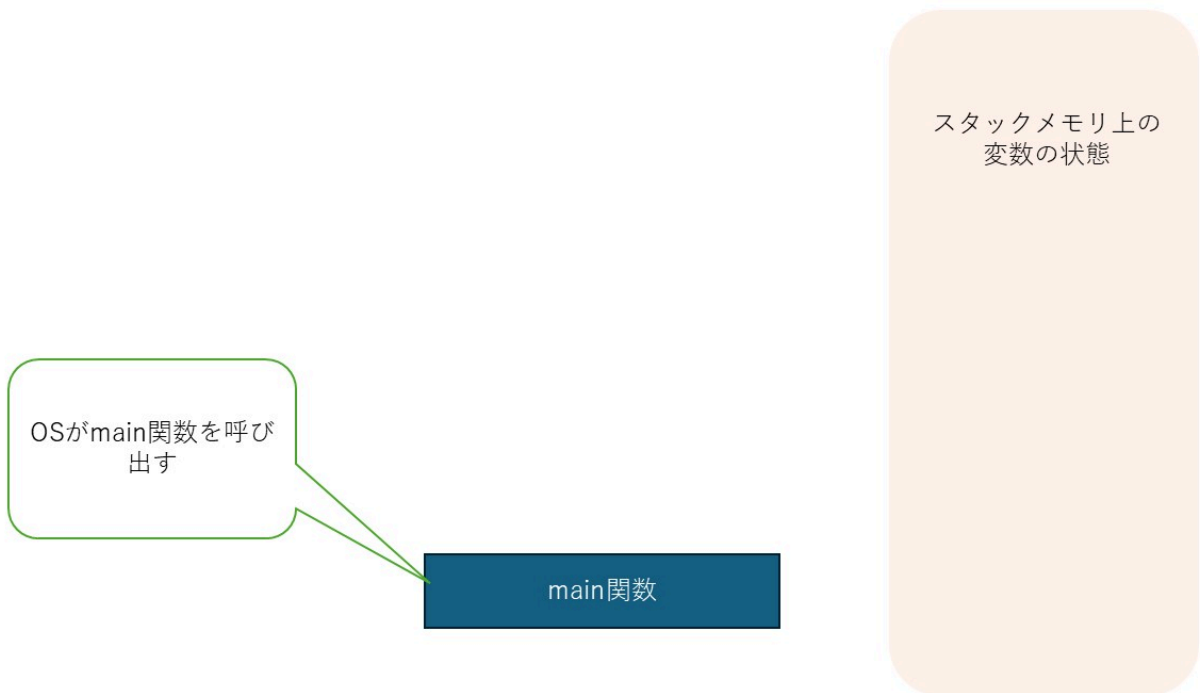
int main() {
    printf("3 times 5 is %d", multiply(3, 5));
    return 0;
}

```

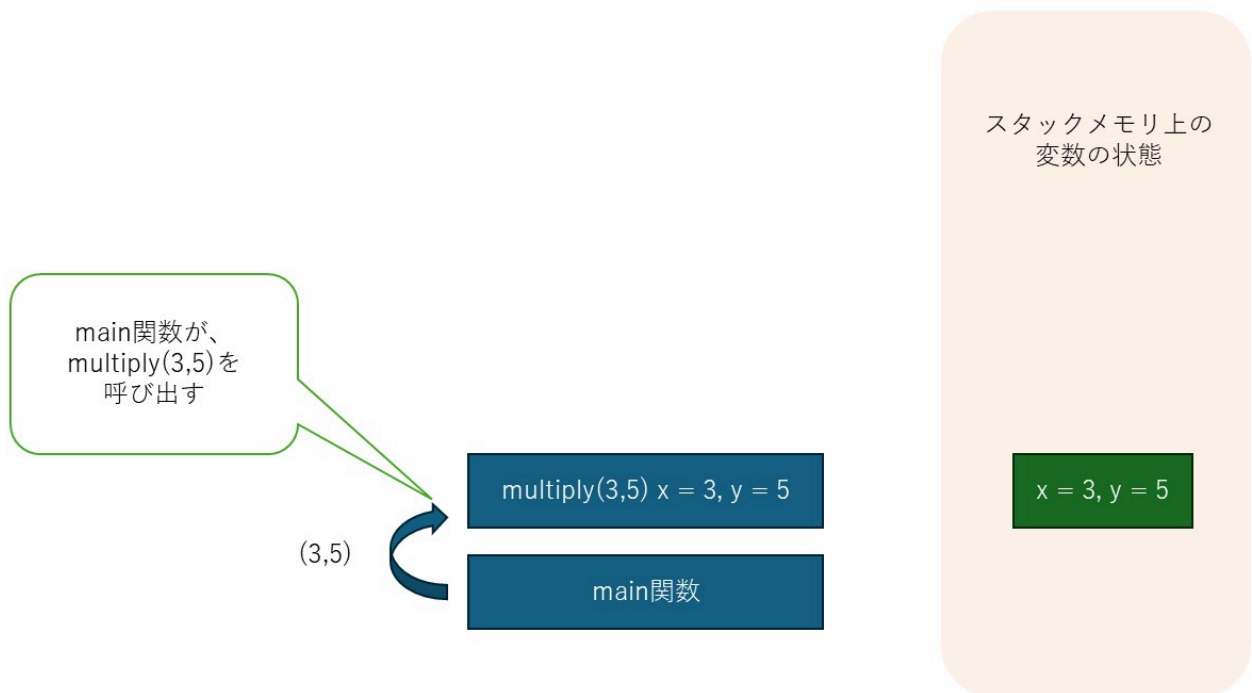
このコードで `multiply(3, 5)` が呼び出された際の処理の流れを、呼び出しスタックの動きに注目して見ていきましょう。

【フェーズ1：再帰呼び出し（スタックに積まれる処理）】

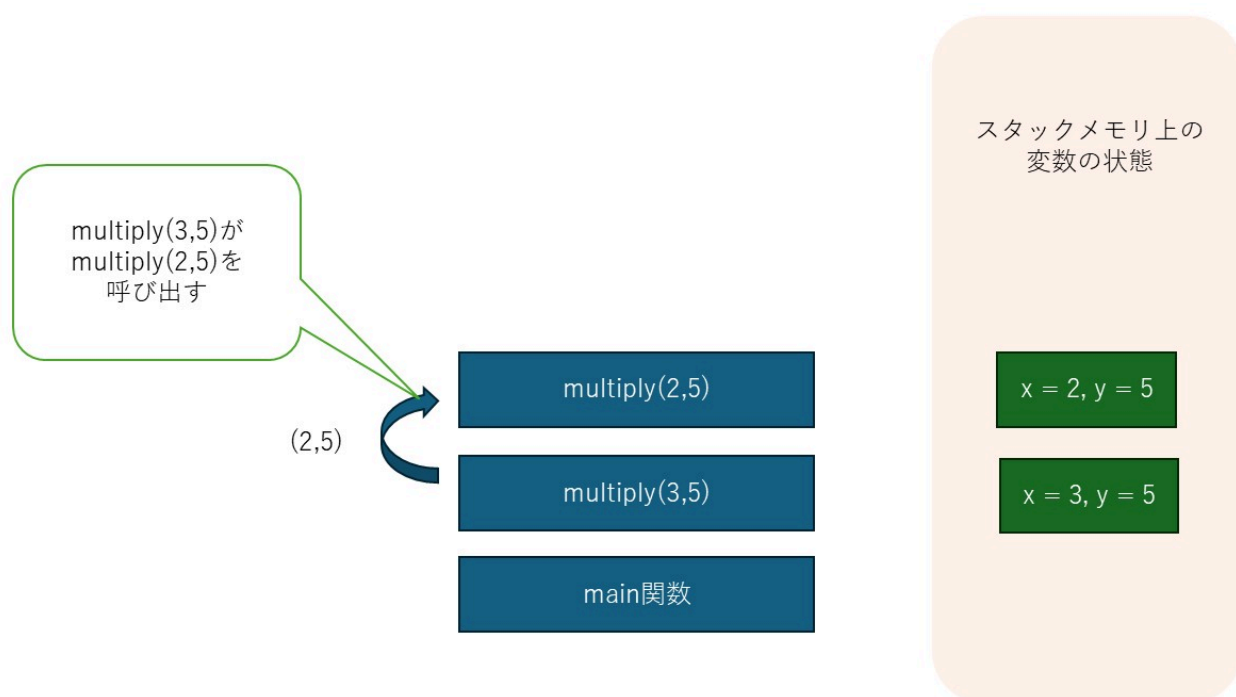
1. OSがmain関数を呼び出す



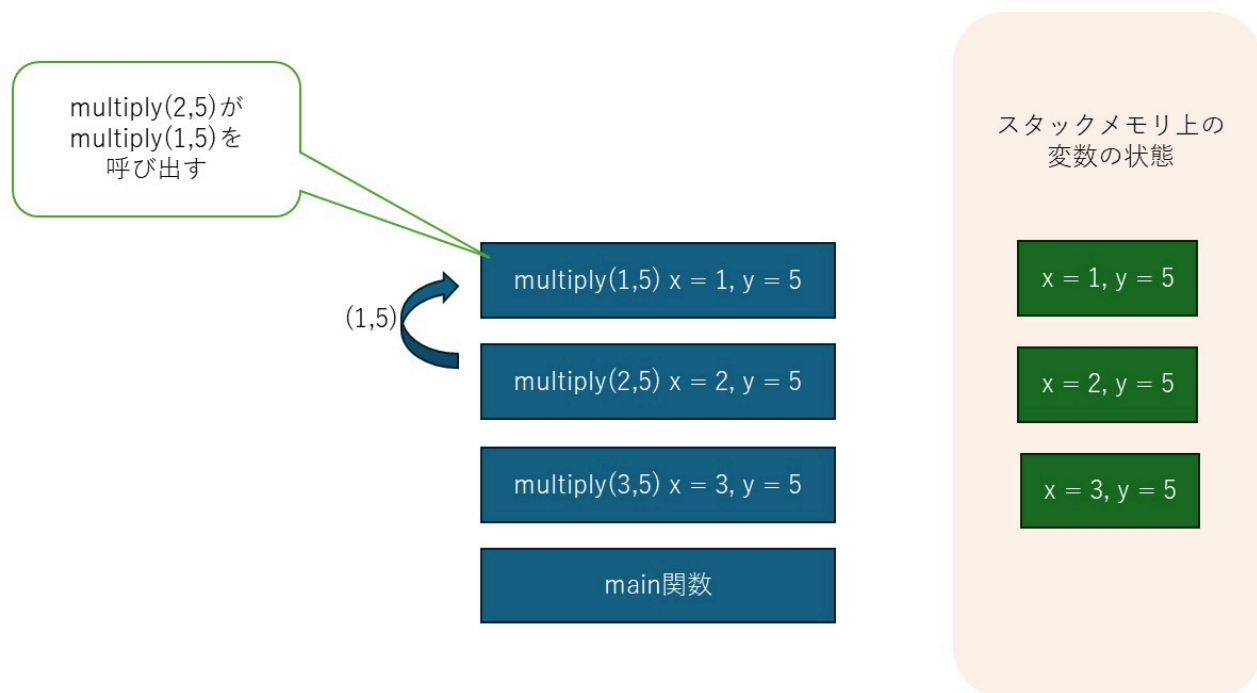
2. main関数がmultiply(3,5)を呼び出し:



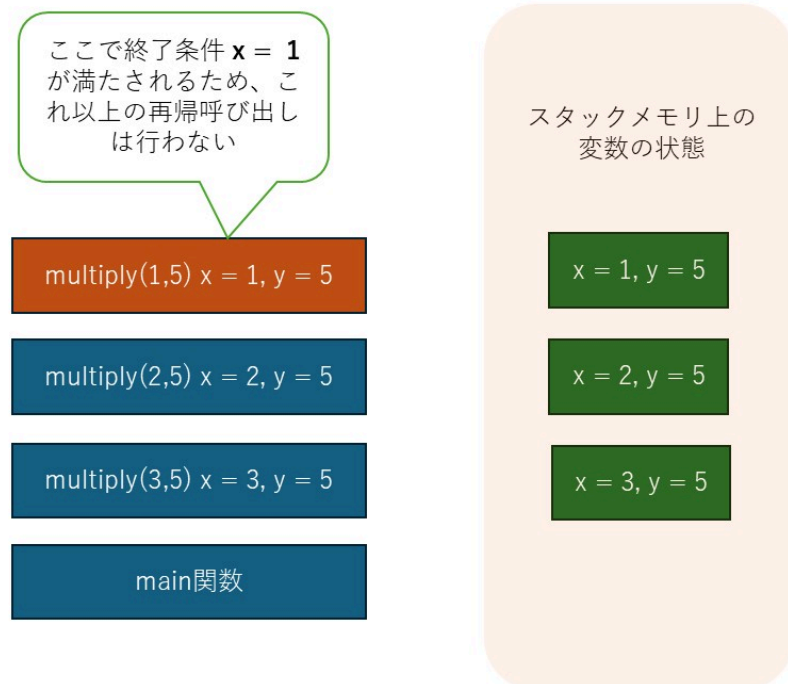
3. `multiply(3, 5)`が`multiply(2, 5)`を呼び出し: `x` は 3。 `x > 1` の条件が真となり、 `5 + multiply(2, 5)` の結果を返すために待機します。 `multiply(2, 5)` の計算が終わるまで、この処理は中断されます。



4. `multiply(2, 5)`が`multiply(1, 5)`を呼び出し: `x` は 2。 `x > 1` の条件が真となり、 `5 + multiply(1, 5)` の結果を返すために待機します。

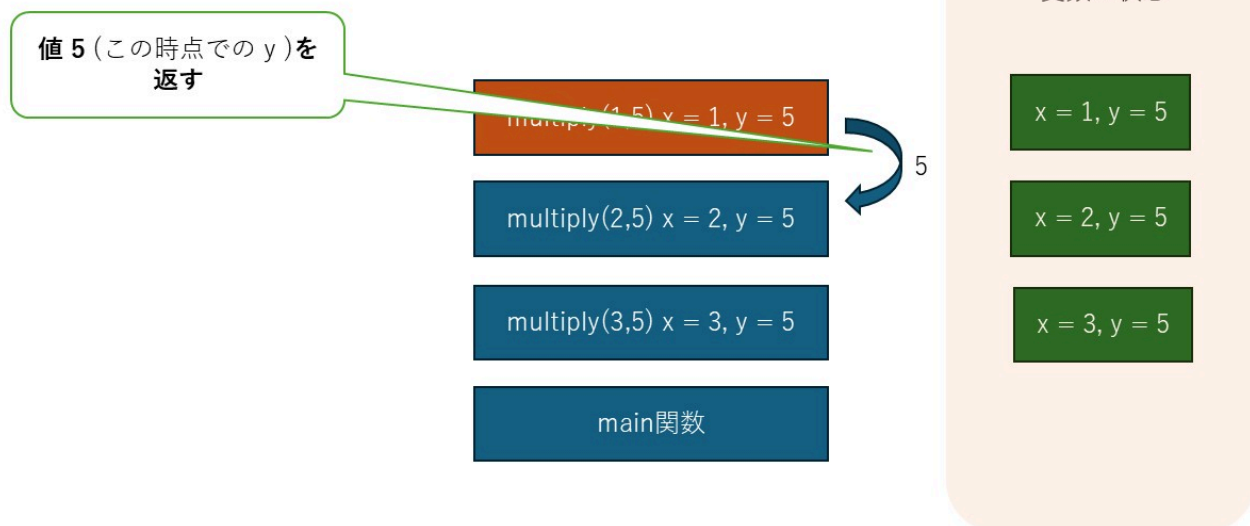


4. `multiply(1, 5)`が呼び出し: `x` は 1。ここでついに `x == 1` の終了条件が真となります。この関数はこれ以上自分自身を呼び出しません。

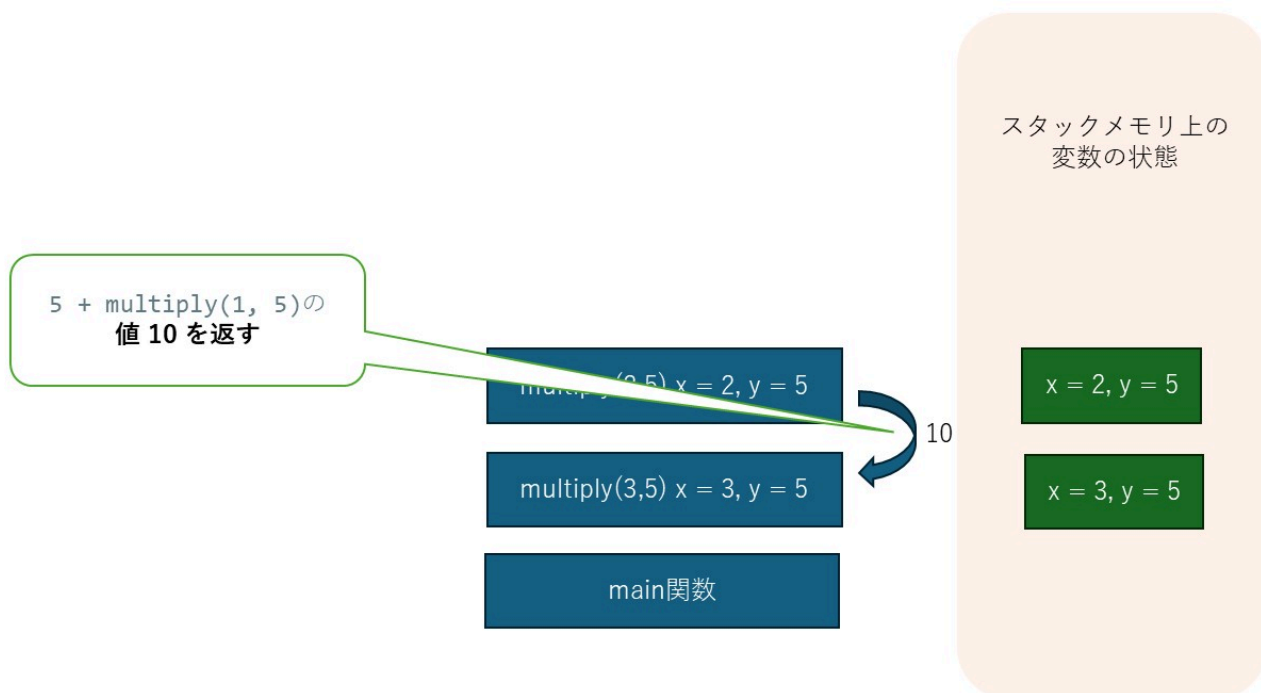


【フェーズ2：スタックの巻き戻しと計算（解決）】

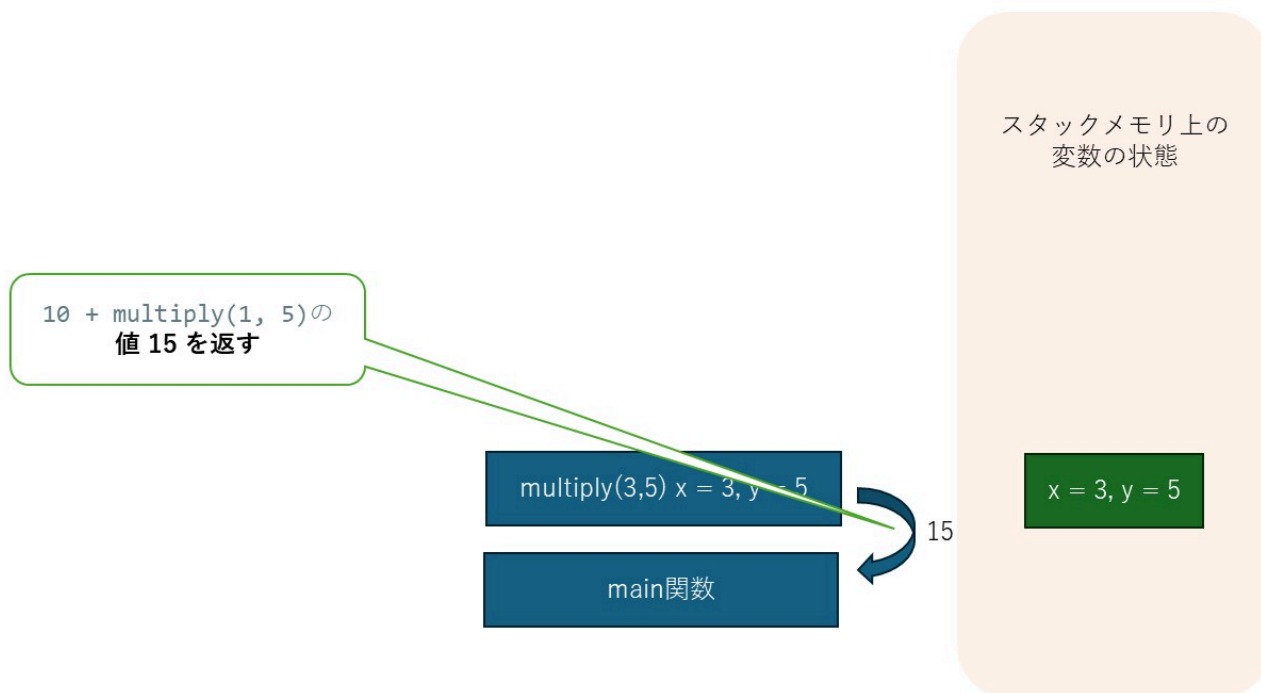
1. `multiply(1, 5)` は値 5 を、`multiply(2, 5)` に返します。



2. `multiply(1, 5)` が返した 5 は、ステップ2で待機していた `multiply(2, 5)` の計算 $5 + \text{multiply}(1, 5)$ に渡されます。計算は $5 + 5$ となり、結果の 10 が `multiply(3, 5)` に返されます。



3. `multiply(2, 5)` が返した 10 は、ステップ1で待機していた `multiply(3, 5)` の計算 `5 + multiply(2, 5)` に渡されます。計算は `5 + 10` となり、最終結果の 15 が main 関数に返されます。



4. main関数が最終結果を表示します。

main関数が
"3 times 5 is 15"
をプリントして
終了する

main関数

スタックメモリ上の
変数の状態

このように、再帰はまず終了条件に達するまで深く潜り（フェーズ1）、そこから各階層の計算を解決しながら浮上してくる（フェーズ2）イメージです。if (x == 1) がこの深い潜水を止める 終了条件 であり、return y + multiply(x-1, y); がより深く潜るための 再帰ステップ となっています。また、xが0の場合のシナリオも考慮されており、堅牢な関数となっています。